# 💻 Desktop 2D Plane Tracking and Grasping AprilTags

## 1. Course Overview

The function implemented in this lesson can be simply understood as: **vehicle and robotic arm follow the AprilTag as it moves, automatically descend to grasp when the block stops**.

Unlike 3D omnidirectional tracking, this lesson constrains the robotic arm end-effector to move within a **fixed-height horizontal top-down plane**. The program uses AprilTag detection to obtain the tag's desktop position in real-time, driving the vehicle and robotic arm to track the AprilTag moving above the table. When the AprilTag stops sliding and remains continuously still for 15 frames, the chassis automatically and smoothly approaches, and the robotic arm vertically descends to execute in-place grasping.

> **Note**
>
> **The tutorial recommends using the accompanying 4×4cm AprilTag cube wooden blocks.**

---

## 2. Preparation and Running

This case requires opening 3 terminals to start the system sequentially:

### 2.1 Start Camera Driver and Kinematics Solver Environment

Run in **Terminal 1**:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

```
jetson@yahboom:~$ ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-16-32-09-195196-yahboom-1117638
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [camera_launch-1]: process started with pid [1117670]
[INFO] [kinemarics_dofbot-2]: process started with pid [1117672]
[INFO] [Ackman_driver_A1P-3]: process started with pid [1117674]
[kinemarics_dofbot-2] ----------------
[kinemarics_dofbot-2] fk
[kinemarics_dofbot-2] 5
[kinemarics_dofbot-2] ----------------
[camera_launch-1] [INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-16-32-10-139043-yahboom-1117670
[camera_launch-1] [INFO] [launch]: Default logging verbosity is set to INFO
[camera_launch-1] [INFO] [usb_cam_node_exe-1]: process started with pid [1117735]
[camera_launch-1] [usb_cam_node_exe-1] libEGL warning: DRI2: failed to authenticate
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.852521184] [usb_cam]: camera_name value: test_camera
[camera_launch-1] [usb_cam_node_exe-1] [WARN] [1788337930.852749618] [usb_cam]: framerate: 120.000000
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.876670665] [usb_cam]: camera calibration URL: package://usb_cam/config/cam
era_info.yaml
[camera_launch-1] [usb_cam_node_exe-1] [WARN] [1788337930.879330849] [usb_cam]: [test_camera] does not match narrow_stereo in file /
home/jetson/yahboomcar_ros2_ws/software/library_ws/install/usb_cam/share/usb_cam/config/camera_info.yaml
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.953127689] [usb_cam]: Starting 'test_camera' (/dev/video0) at 640x480 via
mmap (mjpeg2rgb) at 120 FPS
[camera_launch-1] [usb_cam_node_exe-1] [swscaler @ 0xaaaad038ffe0] No accelerated colorspace conversion found from yuv422p to rgb24.
[camera_launch-1] [usb_cam_node_exe-1] This device supports the following formats:
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1280 x 720 (60 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1920 x 1080 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1024 x 768 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 640 x 480 (120 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 800 x 600 (60 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1280 x 1024 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 320 x 240 (120 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1280 x 720 (9 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1920 x 1080 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1024 x 768 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 640 x 480 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 800 x 600 (20 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1280 x 1024 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 320 x 240 (30 Hz)
[Ackman_driver_A1P-3] [INFO] [1788337931.035307899] [driver_node]: A1 driver initialized: imu_link=imu_link, pub_hz=20.0, arm_status
_hz=5.0, cmd_vel_timeout=0.50s, pub_odom=True
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.748886319] [usb_cam]: Setting 'white_balance_temperature_auto' to 1
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.749045211] [usb_cam]: Setting 'exposure_auto' to 3
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.760676716] [usb_cam]: Setting 'focus_auto' to 0
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.836160385] [usb_cam]: Timer triggering every 8 ms
[camera_launch-1] [usb_cam_node_exe-1] [mjpeg @ 0xaaaad0382960] No JPEG data found in image
[camera_launch-1] [usb_cam_node_exe-1] Failed to send AVPacket to decode: Invalid data found when processing input
[camera_launch-1] [usb_cam_node_exe-1] Failed to send AVPacket to decode: Invalid data found when processing input
```

## 2.2 Start Robotic Arm Desktop Grasping Action Execution Node

Run in **Terminal 2**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver grasp_desktop

# Or USB camera version
ros2 run dofbot_pro_usb grasp_usb
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb grasp_usb
{'x_offset': 0.009368814795573138, 'y_offset': 0.007718452024610358, 'z_offset': 0.0}
---------------------------
x_offset:  0.009368814795573138
y_offset:  0.007718452024610358
z_offset:  0.0
----------------------
[INFO] [1788337928.183721749] [Grasp_Node]: Service not available, waiting again...
[INFO] [1788337929.186574716] [Grasp_Node]: Service not available, waiting again...
init done
self.CurEndPose:  [0.0938259179959246, 0.0002628454756523636, 0.12303978979136244, -4.489092034631593e-06, 1.3962632447969856, -2.82
89145258287586e-05]
```

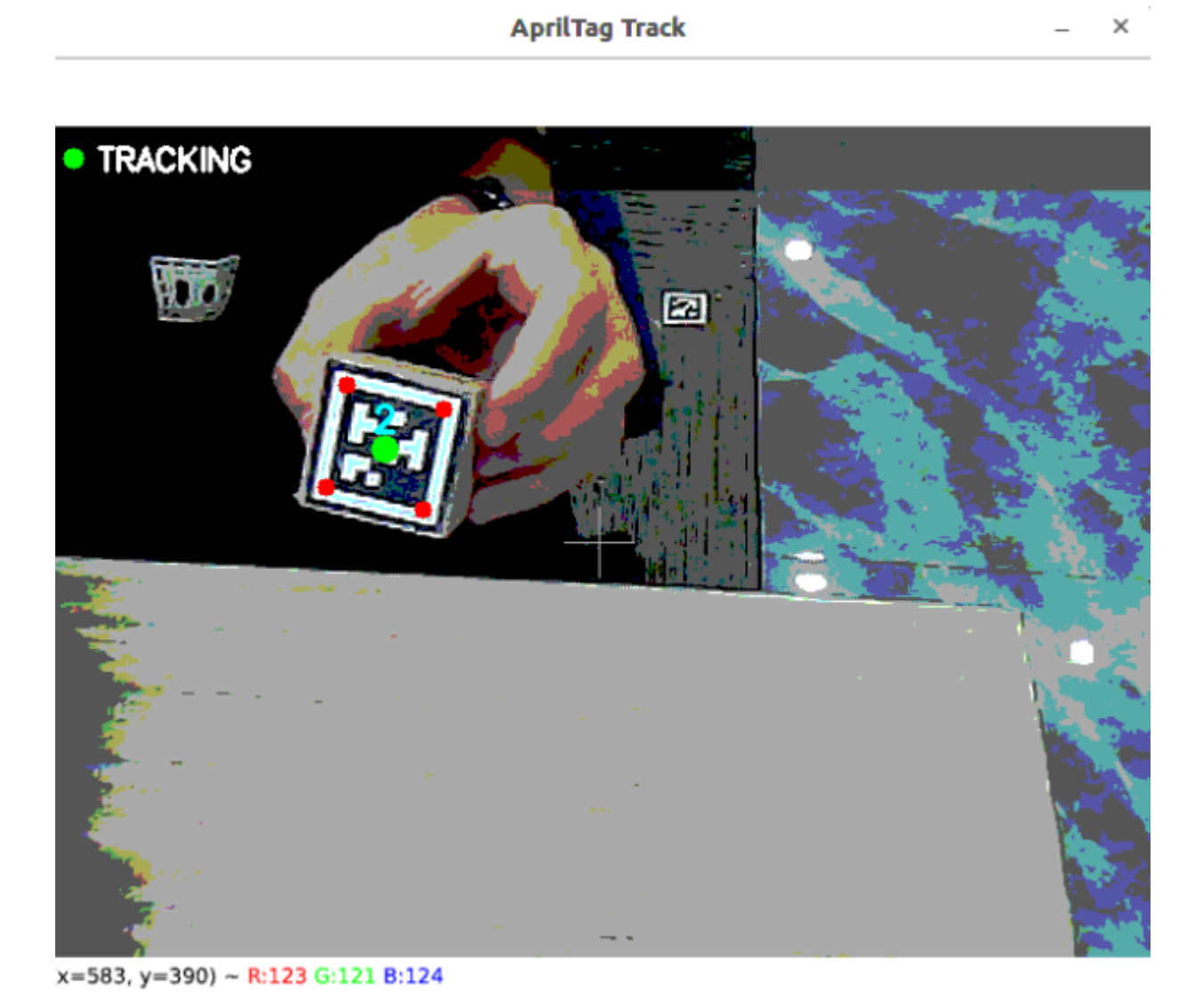
## 2.3 Start Desktop Plane Tracking Node

Run in **Terminal 3**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver apriltag_track_desktop

# Or USB camera version
ros2 run dofbot_pro_usb apriltag_track_desktop
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb apriltag_track_desktop
----------------------
Loaded Offset -> x: 0.009368814795573138, y: 0.007718452024610358, z: 0.0
[INFO] [1788344168.054782117] [ApriltagTrack_node]: tag_size: 0.025 m
```




# 3. Operation Demo and Interaction Flow

1. **Handheld block desktop translation**: Hold the AprilTag block and slide it steadily on the table. The robotic arm end-effector stays at a fixed safe height above the block. The vehicle and robotic arm will track the AprilTag moving above the table;
2. **Stationary detection**:
    - When the block stops sliding, the algorithm starts detecting pixel displacement;
    - When pixel displacement is less than 3 pixels for all 15 consecutive frames (approximately 0.5~1.0 seconds), the system confirms the block has stopped and locks it;
3. **Automatic approach and in-place descent**:
    - Buzzer sounds, chassis automatically and smoothly advances to align with the material;
    - Robotic arm automatically deflects the gripper based on tag rotation angle and vertically descends to grasp;
4. **Motion reset**: After grasping, the robotic arm resets to the initial patrol pose, resuming monitoring and

tracking for the next block movement.

> **Tip**
>
> When moving the block, please **keep it steady**, avoid fast waving causing robotic arm follow overshoot. When stopping, completely release your hand to avoid hand micro-tremors resetting the stationary counter.

---

# 4. Core Source Code Analysis

The system is completed by "desktop plane tracking decision" and "IK execution" two nodes working in coordination:

| Camera Type | Plane Tracking Decision Node | Grasping Action Execution Node |
| --- | --- | --- |
| **USB Monocular Camera** | `dofbot_pro_usb/apriltag_track_desktop.py` | `dofbot_pro_usb/grasp_usb.py` |
| **3D Depth Camera** | `dofbot_pro_driver/apriltag_track_desktop.py` | `dofbot_pro_driver/grasp_desktop.py` |

The core business logic comparison between the two:

| Comparison Dimension | USB Monocular Camera Solution | 3D Depth Camera Solution (Nuwa-HP60C) |
| --- | --- | --- |
| **Tracking Center Reference** | Image optical center `TARGET_X = 341, TARGET_Y = 242` | Image center `TARGET_X = 320, TARGET_Y = 240` |
| **Stationary Detection Threshold** | Continuous 15 frames with displacement less than 3.0 pixels | Continuous 15 frames with displacement less than 3.0 pixels ( `STATIC_THRESHOLD = 3.0` ) |
| **Ranging and Approach Distance** | Pinhole calculated geometric distance, approach to approximately 190mm | Depth matrix directly indexes true physical distance, approach to 205mm ( `GRASP_DIST = 205` ) |
| **Spatial Compensation Dimension** | Superimpose X, Y 2D plane calibration deviation | Synchronously superimpose X, Y, Z 3D omnidirectional calibration compensation ( `offset_value.yaml` ) |

---

## 4.1 Continuous 15 Frames Stationary Anti-Jitter Detection Algorithm

To avoid falsely triggering grasping during block movement, both versions implement highly sensitive stationary detection:

```
# apriltag_track_desktop.py —— Stationary displacement detection
dist_moved = math.hypot(center_x - self.last_tag_x, center_y - self.last_tag_y)

if dist_moved < STATIC_THRESHOLD:  # Displacement less than 3 pixels
    self.static_counter += 1
    if self.static_counter >= STATIC_FRAMES_REQ:  # Continuous stationary reaches 15 frames
        self.adjust_dist = True                   # Activate chassis approach
        self.XY_Track_flag = False                # Lock end-effector plane tracking
else:
    # As long as the block is still moving, immediately reset counter and update reference
coordinates
    self.static_counter = 0
    self.last_tag_x = center_x
    self.last_tag_y = center_y
```

---

## 4.2 Plane PID Following and Chassis Approach

### 1. Fixed-Height Plane Gimbal Following (`XY_track`)

Fix the robotic arm pitch angle, only use base left/right rotation and front/rear link extension to adjust following target, keeping the gripper end-effector sliding parallel at a fixed table height:

```
# apriltag_track_desktop.py —— Plane following
raw_angular_x = self.angular_x_PID.compute(center_x, TARGET_X)
raw_angular_y = self.angular_y_PID.compute(center_y, TARGET_Y)

# Slope limiter smoothing filter
angular_x = self.last_angular_x + max(-self.max_step_change, min(self.max_step_change,
diff_x))

target_j1 = 90  + (10 * angular_x * gain_j1)  # Adjust left/right yaw
target_j2 = 120 + (angular_y * gain_j2)        # Adjust front/rear extension, height remains
constant
```

### 2. Depth Camera Hardware Ranging and Grasping Publishing

After confirming stationary status, chassis drives smoothly toward the target block based on true physical distance. After arriving, solve gripper deflection angle and publish 3D grasping pose:

```
# apriltag_track_desktop.py —— Depth camera grasping publishing
cz = depth_image[int(cy), int(cx)] / 1000.0
if cz != 0:
    vx = int(tag.corners[0][0]) - int(tag.corners[1][0])
    vy = int(tag.corners[0][1]) - int(tag.corners[1][1])
    self.joint5.data = int(compute_joint5(vx, vy))  # Gripper adaptive angle

    pos = AprilTagInfo()
    pos.id = tag.tag_id
    pos.x = float(cx)
    pos.y = float(cy)
```

```
    pos.z = float(cz)

    self.TargetJoint5_pub.publish(self.joint5)
    self.pos_info_pub.publish(pos)  # Trigger grasp_desktop to execute descent
```

After grasping completes, the node resets all status and resets the robotic arm to initial pose, smoothly resuming desktop plane tracking for the next sliding block.

## 4.3 Grasping Offset Tuning (Both Camera Models)

If position deviation occurs during actual grasping (such as missing, knocking over objects, offset left/right, or hitting the table), you can directly modify the spatial offset configuration file for the corresponding camera:

### 1. Where to Modify

Configuration files are located in the `~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_driver/config/` directory. Select based on camera type:

- **3D Depth Camera (Nuwa-HP60C)**: `offset_value.yaml`
- **USB Camera**: `usb_offset_value.yaml`

### 2. How to Modify

The file contains offset parameters in X, Y, Z directions (unit: meters, recommended adjustment per tweak is `0.005`~`0.01`, i.e., 5~10mm):

- `x_offset` **(forward/backward direction, positive toward vehicle front)**:
  - Gripper extends too far and knocks over object **decrease value** (adjust smaller or set negative)
  - Gripper doesn't extend far enough and misses **increase value** (adjust larger)
- `y_offset` **(left/right direction, positive toward vehicle left)**:
  - Gripper deviates right **increase value** (correct left)
  - Gripper deviates left **decrease value** (correct right)
- `z_offset` **(up/down height, positive toward ceiling)**:
  - Gripper too high in air and can't grasp **decrease value** (probe deeper)
  - Gripper hits or scrapes table **increase value** (raise higher)